

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ**

**КУРСОВАЯ РАБОТА
по дисциплине «Программирование»
Тема: Работа с текстом**

Студент гр. 5387

Степанов Л.А.

Преподаватель

Гаврилов А.В.

Санкт-Петербург

2025

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студент: Степанов Л. А.

Группа 5387

Тема работы: Работа с текстом

Исходные данные:

Необходимо разработать консольную утилиту, которая позволяет пользователю выполнять обработку текста и работать с предложениями по заранее определенному набору команд:

0. Вывод предложений после первичной предобработки текста(найти и удалить все повторно встречающиеся предложения (сравнивать их следует посимвольно, но без учета регистра)).
1. Найти все предложения в которых есть слова вида "<число>sec" ("23sec") задающие количество секунд. Для каждого такого предложения посчитать количество секунд и вывести количество полученных часов, минут и секунд в виде строки "hour:min:sec". Выводить сами предложения не нужно. Если в предложении нет слов подобного вида, то для них ничего выводить не следует. Порядок вывода определяется порядком вхождения, например если во всем тексте только три предложения, которые содержат слова данного формата, то вывод будет иметь следующий вид:
5:24:12
0:32:55
4:52:6
2. В каждом предложении удалить первое слово. Если в предложении не останется слов, то его надо удалить.
3. Удалить все предложения у которых длины всех слов одинаковые.
4. Отсортировать предложения по уменьшению количества строчных букв в нем. В случае равенства сохранить порядок вхождения.

Содержание пояснительной записки:

«Содержание», «Введение», «Заключение», «Список использованных источников», «Приложение А. Пример работы программы».

Предполагаемый объем пояснительной записки:

Не менее 25 страниц.

Дата выдачи задания: 16.10.2025

Дата сдачи реферата 03.12.2025

Дата защиты реферата: 05.12.2025

Студент гр. 5387

Степанов Л.А.

Преподаватель

Гаврилов А.В.

АННОТАЦИЯ

Данная курсовая работа направлена на создание консольной программы, предназначенной для обработки предложений, которые вводит пользователь. Исследуются способы хранения текстовой информации, методы фильтрации и преобразования строковых данных. В программе реализован функционал для устранения дублирующихся предложений, определения слов и вычисления частоты их употребления, переворачивания символов внутри слов, отбора предложений, содержащих запятые, а также упорядочивания по коду пятого символа. При разработке применялись динамическое выделение памяти, работа с указателями, создание пользовательских функций и оригинальные алгоритмы для манипуляций со строками. Результаты подтверждают успешное выполнение всех заданных требований.

SUMMARY

This course work focuses on creating a console-based program designed to process user-inputted sentences. The study examines text storage techniques, filtering approaches, and string transformation methods. The application implements functionality for removing duplicate sentences, identifying words and calculating their usage frequency, reversing characters within words, selecting sentences containing commas, and ordering by the fifth character's code. The development process involved dynamic memory allocation, pointer manipulation, user-defined functions, and original string-handling algorithms. The outcomes confirm successful fulfillment of all specified requirements.

СОДЕРЖАНИЕ

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ	2
АННОТАЦИЯ	4
SUMMARY.....	4
СОДЕРЖАНИЕ	5

ВВЕДЕНИЕ

Целью работы является создание программного средства для обработки текстовых данными, организованными как совокупность предложений.

В рамках работы решается ряд задач: обеспечение правильного ввода и сохранения текста; разработка механизмов исключения дублирующихся предложений; осуществление различных манипуляций с текстом на основе пользовательского выбора, таких как определение количества слов, инверсия символов в словах, отбор предложений согласно заданному условию и упорядочивание по значению кода символа.

Для достижения поставленных целей используются механизмы динамического распределения памяти, авторские алгоритмы работы со строками и функции сортировки, а также применяется структурный подход при построении программы.

1. ФУНКЦИИ

1.1 Функция `main.c`

Функция `main` отображает сведения о варианте курсовой работы, получает от пользователя номер режима и инициирует соответствующую процедуру обработки текста. После считывания номера опции выполняется очистка входного буфера с помощью `getchar()`.

При выборе режимов 0–4 программа сначала получает текст посредством `read_arr`, затем производит предварительную обработку через `pred_obrabotka` (удаление повторяющихся предложений). После этого вызывается функция, соответствующая выбранному режиму: вывод обработанного текста, подсчёт и суммирование временных меток, удаление первого слова каждого предложения, исключение предложений с одинаковой длиной слов или сортировка по количеству строчных букв. Результат выводится на экран при наличии данных. При возникновении ошибок на любом этапе функция возвращает код ошибки, и программа выводит соответствующее сообщение.

Режим 5 отображает справочную информацию о доступных командах без чтения текста. По завершении работы каждой функции выделенная динамическая память освобождается вызовом `free`, после чего функция возвращает код завершения.

1.2 Функция считывания текста

- **`read_arr`**

Функция `read_arr` отвечает за считывание полного текста из стандартного ввода. Чтение осуществляется посимвольно, а увеличение буфера выполняется при необходимости путём удвоения его размера. Ввод завершается при обнаружении двух последовательных символов новой строки либо при достижении конца файла. В результате формируется динамическая строка, содержащая весь введённый текст. При ошибке выделения памяти возвращается `NULL`.

- **`split_sentence`**

Функция `split_sentence` разбивает исходный текст на отдельные предложения, используя точку в качестве разделителя. Она создаёт копию входной строки и применяет `strtok` для последовательного извлечения фрагментов. Ведущие пробелы и табуляции удаляются, к каждому предложению добавляется точка в конце. Полученные строки сохраняются в динамическом массиве указателей, размер которого увеличивается по мере необходимости. При любой ошибке происходит корректное освобождение уже выделенных строк.

- **`join_arr`**

Функция `join_arr` объединяет массив строк в одну строку с указанным разделителем. Она предварительно вычисляет общую длину результата, выделяет память и последовательно копирует элементы с разделителями между ними. Используется для сборки обработанных предложений обратно в текст.

- **trim**

Функция trim удаляет пробельные символы с обоих концов строки. Она последовательно перемещает указатель вперёд, игнорируя пробелы и табуляции в начале, затем находит конец значимого содержимого и устанавливает терминатор. Применяется при сравнении предложений для устранения влияния лишних пробелов.

- **trim_punct**

Функция trim_punct аналогична trim, но удаляет знаки пунктуации вместо пробельных символов. Используется при обработке слов для корректного сравнения длин.

- **remove_sentence**

Функция remove_sentence удаляет предложение из массива по указанному индексу. Она освобождает память удаляемого элемента, сдвигает все последующие элементы на одну позицию влево и уменьшает счётчик количества предложений.

- **swap**

Функция swap выполняет обмен двух указателей на строки. Используется в алгоритмах сортировки для перестановки элементов массива.

- **count_islower**

Функция count_islower подсчитывает количество строчных букв в переданной строке. Она проходит по всем символам и увеличивает счётчик при обнаружении символа нижнего регистра. Применяется в команде сортировки.

1.3 Функции выполняющие команды и вспомогательные функции

- **pred_obrabotka (pred_obrabotka.c)**

Функция pred_obrabotka удаляет повторяющиеся предложения из текста. Она разбивает входную строку на массив предложений с помощью split_sentence, затем сравнивает каждое предложение с последующими, используя strcasecmp для сравнения без учёта регистра. Найденные дубликаты удаляются вызовом remove_sentence. После этого формируется новая строка путём последовательной конкатенации оставшихся предложений с добавлением пробелов между ними. Вызывается перед выполнением любой команды обработки.

- **print_sum_time (one.c)**

Функция `print_sum_time` выполняет поиск временных меток в формате «число sec» и выводит их в формате часы:минуты:секунды. Она использует `strstr` для нахождения вхождений «sec», затем извлекает предшествующие цифры, формируя числовое значение. Каждое найденное значение конвертируется и выводится отдельной строкой.

- **`del_first_sentence (two.c)`**

Функция `del_first_sentence` удаляет первое слово в каждом предложении. Она разбивает текст на предложения, затем для каждого находит первый пробел и сдвигает содержимое строки влево, удаляя начальный фрагмент. Предложения, состоящие из одного слова или ставшие пустыми после удаления, исключаются из результата. Обработанные предложения собираются обратно в строку.

- **`remove_equal_length_sentences (three.c)`**

Функция `remove_equal_length_sentences` удаляет предложения, в которых все слова имеют одинаковую длину. Для каждого предложения создаётся копия, которая разбивается на слова с помощью `strtok`. Длины слов сравниваются после удаления пунктуации функцией `trim_punct`. Если все слова имеют равную длину и слов больше одного, предложение удаляется из массива. Результат собирается с помощью `join_arr`.

- **`sort_by_islower (four.c)`**

Функция `sort_by_islower` сортирует предложения по убыванию количества строчных букв. Она использует вспомогательную структуру `SentenceInfo` для хранения указателя на строку, исходного индекса и подсчитанного количества строчных символов. Сортировка выполняется стандартной функцией `qsort` с компаратором `compare_sentences`, который сначала сравнивает по количеству строчных букв, а при равенстве сохраняет исходный порядок. Перед обработкой символы новой строки заменяются пробелами функцией `remove_newlines`.

- **`compare_sentences (four.c)`**

Функция `compare_sentences` служит компаратором для `qsort`. Она сравнивает две структуры `SentenceInfo`: при различном количестве строчных букв возвращает разность для сортировки по убыванию, при равенстве сравнивает исходные индексы для сохранения стабильности сортировки.

- **`remove_newlines (four.c)`**

Статическая функция `remove_newlines` заменяет символы перевода строки и возврата каретки на пробелы. Применяется для нормализации текста перед разбиением на предложения.

2. СТРУКТУРЫ

2.1 Структура SentenceInfo

Она используется для сортировки предложений по количеству строчных букв. Структура хранит указатель на строку предложения, его исходный индекс в массиве (для сохранения стабильности сортировки) и подсчитанное количество строчных символов.

3. СБОРКА ПРОЕКТА

Makefile проекта расположен в корневой директории и обеспечивает автоматическую компиляцию исходных файлов `main.c`, `slon_tools.c`, `pred_obrabotka.c`, `one.c`, `two.c`, `three.c` и `four.c`.

Проект имеет организованную структуру: исходные файлы находятся в каталоге `src`, заголовочные файлы — в каталоге `include`, объектные файлы создаются в каталоге `obj`.

Поддерживаются следующие команды:

1. `make / make all` — выполняет сборку проекта, компилируя все модули из `src` в объектные файлы в `obj` и формируя исполняемый файл `sw`.
2. `make clean` — удаляет каталог с объектными файлами и собранный бинарный файл, позволяя выполнить полную пересборку.
3. `make re` — последовательно выполняет очистку и повторную сборку проекта.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана консольная программа на языке С, предназначенная для обработки текстовой информации и выполнения над ней ряда операций. Основной целью проекта являлась реализация набора алгоритмов для анализа, преобразования и фильтрации предложений с использованием различных режимов обработки, выбираемых пользователем. Для достижения поставленной цели были применены следующие методы:

1. Реализация динамического ввода текста с автоматическим расширением буфера и последующим разбором строки на предложения.
2. Построение алгоритмов удаления повторяющихся предложений с использованием сравнения строк без учёта регистра.
3. Разработка механизмов анализа текста: поиск временных меток в формате «число sec» с конвертацией в часы:минуты:секунды, удаление первого слова в предложениях и фильтрация предложений по равенству длин слов.
4. Создание алгоритма сортировки предложений по количеству строчных букв с использованием структуры SentenceInfo и стандартной функции qsort.
5. Организация модульной структуры проекта, разделяющей ввод текста, предварительную обработку и команды по отдельным файлам (main.c, slon_tools.c, pred_obrabotka.c, one.c, two.c, three.c, four.c), что улучшает читаемость и расширяемость кода.

Ключевые результаты:

1. Программа корректно обрабатывает все предусмотренные режимы (0–5).
2. Все функции успешно реализованы и соответствуют требованиям.
3. Обеспечена модульность и надёжность программного кода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Керниган Б., Ритчи Д. Язык программирования С. — 2-е изд. — М.: Вильямс, 2016. — 304 с.
2. ISO/IEC 9899:2011. Programming languages — C. — International Organization for Standardization, 2011.
3. Прата С. Язык программирования С. Лекции и упражнения. — 6-е изд. — М.: Вильямс, 2015. — 928 с.
4. GNU C Library Reference Manual. — URL: <https://www.gnu.org/software/libc/manual/> (дата обращения: 2025).

ПРИЛОЖЕНИЕ А

ПРИМЕР РАБОТЫ ПРОГРАММЫ

В начале каждого вывода подразумевается вывод информации вида:

“Course work for option 4.11, created by Stepanov Leonid.”

Таблица 1 – работа команды 0:

Входные данные	Выходные данные
<i>Hello. Repeat. RePeaT. How'are you. Ok. oK. Bye.</i>	<i>Hello. How'are you. Bye.</i>

Таблица 2 – работа команды 1:

Входные данные	Выходные данные
<i>1 I don't want to dopsa 1sec, 123sec. I want to eat and sleep for 9999sec.</i>	<i>0:0:1 0:2:3 2:46:39</i>

Таблица 3 – работа команды 2:

Входные данные	Выходные данные
<i>2 Hello, I want sleep evry time. Ok. Third sentance.</i>	<i>I want sleep evry time. sentance.</i>

Таблица 4 – работа команды 3:

Входные данные	Выходные данные
<i>3 1234567890. 1234567890. 12345. 12345. 123. 1234.</i>	<i>123. 1234.</i>

Таблица 5 – работа команды 4:

Входные данные	Выходные данные
<i>4 1 HELLo WORLD. 3 HELLo WORLD. 5 Hello wORLD. 2 HELLO WORLD.</i>	<i>5 Hello wORLD. 3 HELLo WORLD. 2 HELLO WORLD. 1 HELLo WORLD.</i>

Таблица 6 – работа команды 5:

Входные данные	Выходные данные
5	0. Вывод текста после первичной обработки. 1. Найти предложения с <число>sec, вывести сумму времени. 2. Удалить первое слово в каждом предложении. 3. Удалить предложения с одинаковой длиной слов. 4. Сортировать по убыванию строчных букв (qsort). 5. Справка.

Таблица 7 – подан несуществующей номер команды:

Входные данные	Выходные данные
7 Error: Введите номер от 0 до 5	

ПРИЛОЖЕНИЕ Б

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.c:

```
#include <stdio.h>
#include <stdlib.h>

#include "slon_tools.h"
#include "pred_obrabotka.h"
#include "one.h"
#include "two.h"
#include "three.h"
#include "four.h"

int zero();
int one();
int two();
int three();
int four();
int five();

int main(){
    printf("Course work for option 4.11, created by Stepanov Leonid.\n");

    int option;
    if (scanf("%d", &option) != 1) {
        printf("Error: Не удалось считать номер команды\n");
        return 0;
    }

    getchar();

    int res = 0;
    switch(option){
        case 0:
            res = zero();
            break;
        case 1:
            res = one();
            break;
        case 2:
            res = two();
            break;
        case 3:
            res = three();
            break;
        case 4:
            res = four();
            break;
        case 5:
            res = five();
```



```

        break;
    default:
        printf("Error: Введите номер от 0 до 5\n");
        return 0;
    }

    if (res != 0) {
        printf("Error: Ошибка при выполнении функции или чтении данных\n");
    }

    return 0;
}

int zero(){
    char *text = read_arr();
    if (!text) return 1;
    text = pred_obrabotka(text);
    if (!text) return 1;

    printf("%s\n", text);

    free(text);
    return 0;
}

int one(){
    char *text = read_arr();
    if (!text) return 1;
    text = pred_obrabotka(text);
    if (!text) return 1;

    print_sum_time(text);

    free(text);
    return 0;
}

int two(){
    char *text = read_arr();
    if (!text) return 1;
    text = pred_obrabotka(text);
    if (!text) return 1;

    text = del_first_sentence(text);
    if (text) printf("%s\n", text);

    free(text);
    return 0;
}

int three(){

```

```

    char *text = read_arr();
    if (!text) return 1;
    text = pred_obrabotka(text);
    if (!text) return 1;

    text = remove_equal_length_sentences(text);
    if (text) printf("%s\n", text);

    free(text);
    return 0;
}

int four(){
    char *text = read_arr();
    if (!text) return 1;
    text = pred_obrabotka(text);
    if (!text) return 1;

    text = sort_by_islower(text);
    if (text) printf("%s\n", text);

    free(text);
    return 0;
}

int five(){
    printf("0. Вывод текста после первичной обработки.\n");
    printf("1. Найти предложения с <число>сек, вывести сумму времени.\n");
    printf("2. Удалить первое слово в каждом предложении.\n");
    printf("3. Удалить предложения с одинаковой длиной слов.\n");
    printf("4. Сортировать по убыванию строчных букв (qsort).\n");
    printf("5. Справка.\n");
    return 0;
}

```

Файл one.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#include "one.h"

void print_sum_time(char *text){
    long long sum_sec = 0;

    char *pos = text;
    char *needle = "sec";

```

```

while ((pos = strstr(pos, needle)) != NULL) {
    int end = (int)(pos - text) - 1;
    int start = end;

    while (start >= 0 && isdigit(text[start])){
        start--;
    }
    start++;

    if (start <= end){
        char num_buf[64];
        int len = end - start + 1;
        if (len > 0 && len < 63) {
            strncpy(num_buf, text + start, len);
            num_buf[len] = '\0';

            long val = atol(num_buf);
            sum_sec += val;

            long h = val / 3600;
            long m = (val % 3600) / 60;
            long s = val % 60;
            printf("%ld:%ld:%ld\n", h, m, s);
        }
    }
    pos += 3;
}
}

```

Файл two.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "two.h"
#include "slon_tools.h"

char *del_first_sentence(char *str){
    size_t len;
    char **arr = split_sentence(str, &len);

    for (size_t i = 0; i < len; i++) {
        char *s = arr[i];
        if (strchr(s, ' ') == NULL) {
            memmove(&arr[i], &arr[i+1], (len - i - 1) * sizeof(char*));
            len--;
            i--;
            continue;
        }
    }
}

```

```

        for (size_t y = 0; s[y] != '\0'; y++) {
            if (s[y] == ' ') {
                memmove(s, s + y + 1, strlen(s + y + 1) + 1);
                if(*trim(s) == '\0'){
                    memmove(&arr[i], &arr[i+1], (len - i - 1) * sizeof(char*));
                }
                break;
            }
        }
    }

    size_t size = 64;
    size_t len_done = 0;
    char *arr_done = malloc(size);
    if(!arr_done) return NULL;

    for(size_t i = 0; i < len; i++){
        size_t slen = strlen(arr[i]);

        while(len_done + slen + 2 > size){
            size *= 2;
            char *tmp = realloc(arr_done, size);
            if(!tmp){
                free(arr_done);
                return NULL;
            }
            arr_done = tmp;
        }

        memcpy(arr_done + len_done, arr[i], slen);
        len_done += slen;

        if(i + 1 < len && arr[i][slen - 1] == '.'){
            arr_done[len_done++] = '\n';
        }
    }

    arr_done[len_done] = '\0';

    for(size_t i = 0; i < len; i++) free(arr[i]);
    free(arr);

    return arr_done;
}

```

Файл tree.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

```

```

#include "slon_tools.h"
#include "three.h"

char *remove_equal_length_sentences(char *str) {
    if (!str) return NULL;

    for (char *p = str; *p; p++) {
        if (*p == '\n' || *p == '\r') *p = ' ';
    }

    size_t len;
    char **arr = split_sentence(str, &len);
    if (!arr) return NULL;

    size_t i = 0;
    while (i < len) {
        char *sentence = strdup(arr[i]);
        if (!sentence) {
            i++;
            continue;
        }

        size_t slen = strlen(sentence);
        if (slen > 0 && sentence[slen-1] == '.') {
            sentence[slen-1] = '\0';
        }

        char *saveptr;
        char *token = strtok_r(sentence, " \t", &saveptr);
        int all_equal = 1;
        size_t first_len = 0;
        int word_count = 0;

        while (token != NULL) {
            size_t wlen = 0;
            for (char *p = token; *p; p++) {
                if (isalpha((unsigned char)*p)) {
                    wlen++;
                }
            }

            if (wlen > 0) {
                if (word_count == 0) {
                    first_len = wlen;
                } else if (wlen != first_len) {
                    all_equal = 0;
                }
                word_count++;
            }
        }
    }
}

```

```

        token = strtok_r(NULL, " \\t", &saveptr);
    }

    free(sentence);

    if (all_equal && word_count > 1) {
        remove_sentence(arr, &len, i);
    } else {
        i++;
    }
}

char *result = join_arr(arr, len, "\\n");

for (size_t j = 0; j < len; j++)
    free(arr[j]);
free(arr);

return result;
}

```

Файл four.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#include "four.h"
#include "slon_tools.h"

typedef struct {
    char *str;
    int original_index;
    int lower_count;
} SentenceInfo;

int compare_sentences(const void *a, const void *b) {
    SentenceInfo *sa = (SentenceInfo *)a;
    SentenceInfo *sb = (SentenceInfo *)b;

    if (sb->lower_count != sa->lower_count) {
        return sb->lower_count - sa->lower_count;
    }
    return sa->original_index - sb->original_index;
}

static void remove_newlines(char *s) {
    for (char *p = s; *p; p++) {
        if (*p == '\\n' || *p == '\\r')

```

```

        *p = ' ';
    }
}

char *sort_by_islower(char *str){
    if (!str) return NULL;

    remove_newlines(str);

    size_t len = 0;
    char **arr = split_sentence(str, &len);
    if (!arr) return NULL;

    SentenceInfo *info = malloc(len * sizeof(SentenceInfo));
    if (!info) {
        for(size_t k=0; k<len; k++) free(arr[k]);
        free(arr);
        return NULL;
    }

    for (size_t i = 0; i < len; i++) {
        info[i].str = arr[i];
        info[i].original_index = (int)i;
        info[i].lower_count = count_islower(arr[i]);
    }

    qsort(info, len, sizeof(SentenceInfo), compare_sentences);

    for (size_t i = 0; i < len; i++) {
        arr[i] = info[i].str;
    }

    char *result = join_arr(arr, len, "\n");

    free(info);
    for (size_t i = 0; i < len; i++)
        free(arr[i]);
    free(arr);

    return result;
}

```

Файл pred_obrabotka.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <ctype.h>

#include "pred_obrabotka.h"

```

```

#include "slon_tools.h"

char *pred_obrabotka(char *str){
    for (char *p = str; *p; p++) {
        if (*p == '\n' || *p == '\r') *p = ' ';
    }

    size_t len;
    char **arr = split_sentence(str, &len);
    if (!arr) return NULL;

    for(size_t i = 0; i < len; i++){
        for(size_t j = i + 1; j < len; j++){
            if(strcasecmp(arr[i], arr[j]) == 0){
                remove_sentence(arr, &len, j);
                j--;
            }
        }
    }

    char *arr_done = join_arr(arr, len, "\n");

    for(size_t i = 0; i < len; i++) free(arr[i]);
    free(arr);

    return arr_done;
}

```

Файл slon_tools.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#include "slon_tools.h"

char *read_arr(){
    size_t size = 64;
    size_t len = 0;
    char *arr = malloc(size);
    if (!arr) return NULL;

    int ch;
    while ((ch = getchar()) != EOF) {
        if (len + 1 >= size) {
            size *= 2;
            char *tmp = realloc(arr, size);
            if (!tmp) {
                free(arr);
                return NULL;
            }
        }
        arr[len++] = ch;
    }
    arr[len] = '\0';
    return arr;
}

```



```

        }
        arr = tmp;
    }

    arr[len++] = (char)ch;

    if (len >= 2 && arr[len-1] == '\n' && arr[len-2] == '\n') {
        len -= 2;
        break;
    }
}

arr[len] = '\0';
return arr;
}

char *join_arr(char **arr, size_t len_arr, char *sep) {
    if (len_arr == 0) {
        char *empty = malloc(1);
        if (empty) empty[0] = '\0';
        return empty;
    }

    size_t total_len = 0;
    size_t sep_len = strlen(sep);

    for (size_t i = 0; i < len_arr; i++) {
        total_len += strlen(arr[i]);
    }
    total_len += sep_len * (len_arr - 1);
    total_len += 1;

    char *res = malloc(total_len);
    if (!res) return NULL;

    char *p = res;
    for (size_t i = 0; i < len_arr; i++) {
        size_t len = strlen(arr[i]);
        memcpy(p, arr[i], len);
        p += len;

        if (i < len_arr - 1 && sep_len > 0) {
            memcpy(p, sep, sep_len);
            p += sep_len;
        }
    }

    *p = '\0';
    return res;
}

```

```

char **split_sentence(char *str, size_t *count){
    size_t arr_size = 8;
    size_t len = 0;
    char **arr = malloc(arr_size * sizeof(char*));
    if (!arr) return NULL;

    char *copy = strdup(str);
    if(!copy){
        free(arr);
        return NULL;
    }

    char *myPtr = strtok(copy, ".");
    while(myPtr != NULL){
        // Пропускаем пробелы в начале
        while(*myPtr == ' ' || *myPtr == '\t' || *myPtr == '\n' || *myPtr == '\r'){
            myPtr++;
        }

        // Пропускаем пустые строки
        size_t myLen = strlen(myPtr);
        if(myLen == 0){
            myPtr = strtok(NULL, ".");
            continue;
        }

        if(len >= arr_size){
            arr_size *= 2;
            char **tmp = realloc(arr, arr_size * sizeof(char*));
            if(!tmp){
                for(size_t i = 0; i < len; i++) free(arr[i]);
                free(arr);
                free(copy);
                return NULL;
            }
            arr = tmp;
        }

        arr[len] = malloc(myLen + 2);
        if(!arr[len]){
            for(size_t i = 0; i < len; i++) free(arr[i]);
            free(arr);
            free(copy);
            return NULL;
        }

        strcpy(arr[len], myPtr);
        arr[len][myLen] = '.';
        arr[len][myLen+1] = '\0';
    }

```

```

        len++;
        myPtr = strtok(NULL, ".");
    }

    free(copy);
    *count = len;
    return arr;
}

char *trim(char *s) {
    while(isspace((char)*s)) s++;
    if(*s == 0) return s;

    char *end = s + strlen(s) - 1;
    while(end > s && isspace((char)*end)) end--;

    *(end+1) = '\0';
    return s;
}

char *trim_punct(char *s) {
    while(ispunct((char)*s)) s++;
    if(*s == 0) return s;

    char *end = s + strlen(s) - 1;
    while((end > s && ispunct((char)*end))) end--;

    *(end+1) = '\0';
    return s;
}

void remove_sentence(char **arr, size_t *count, size_t index){
    if(index >= *count) return;

    free(arr[index]);
    for(size_t i = index; i < *count - 1; i++){
        arr[i] = arr[i+1];
    }
    (*count)--;
}

void swap(char **arr1, char **arr2){
    char *buffer = *arr1;
    *arr1 = *arr2;
    *arr2 = buffer;
}

int count_islower(const char *str) {
    int counter = 0;
    for (size_t i = 0; str[i] != '\0'; i++) {
        if (islower((unsigned char)str[i])) counter++;
    }
}

```

```
}  
    return counter;  
}
```

Файл one.h

```
void print_sum_time(char *text);
```

Файл two.h

```
char *del_first_sentence(char *str);
```

Файл three.h

```
char *remove_equal_length_sentences(char *str);
```

Файл four.h

```
char *sort_by_islower(char *str);
```

Файл slon_tools.h

```
char *read_arr();  
char *join_arr(char **arr, size_t len_arr, char *sep);  
char **split_sentence(char *str, size_t *count);  
char *trim(char *s);  
char *trim_punct(char *s);  
int count_islower(const char *str);  
void remove_sentence(char **arr, size_t *count, size_t index);
```

Файл pred_obrabotka.h

```
char *pred_obrabotka(char *str);
```